ITSAR2 313 – SYSTEM INTEGRATION AND ARCHICTECTURE 2
LABORATORY REPORT: EXERCISE 1 MONOLITH VS MICROSERVICE

Delicano, Rovin Destajo; Herman, Angel Mae Arnaiz; Olmedo, Jirah Canillo; Prado, Jhona Mae Octavio; Sudayon, Karyl Ann Delmita
Information Technology, College of Computer Studies
Carlos Hilado Memorial State University

Abstract

This is Student Course System as a laboratory exercise involved two different types of architecture which are monolithic and microservice architecture. The monolithic architecture built in Node.js and Express with Vanilla JavaScript, and for the microservices architecture built in Python Flask with three different independent services: Student Service, Course Service and Enrollment Service each with own SQLite database. This exercise demonstrates the practical differences between two different software architecture in terms of development, complexity, scalability, deployment and service communication.

I. INTRODUCTION

Software architecture defines how a system is structure and how its components connect to each other. Monolithic application is built with a single, unified units where all features, user interface, business logic, and data accessibility coupled and deployed together. This simplifies initial development and testing; difficulties may arise as the system grows.

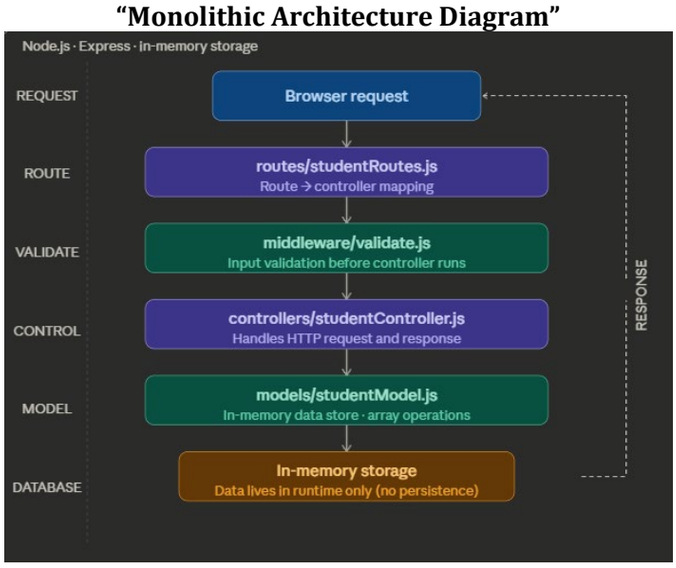
Contrasting to this, microservice architecture decomposes an application into small, independently deployable services that communicates over network, this typically uses HTTP REST APIs. Each is in charge for a specific business domain, maintains own database, and can be developed, deployed and scaled independently. This approach aligns with the principles of separation of concerns and single responsibility.

In this exercise aimed to provide experience on both architectures through Student Course System where it covers student management, course management, and enrollment features. This displays the trade-offs between two architectures and demonstrate proper API design, error handling, and HTTP status code usage.

II. METHODOLOGY / IMPLEMENTATION

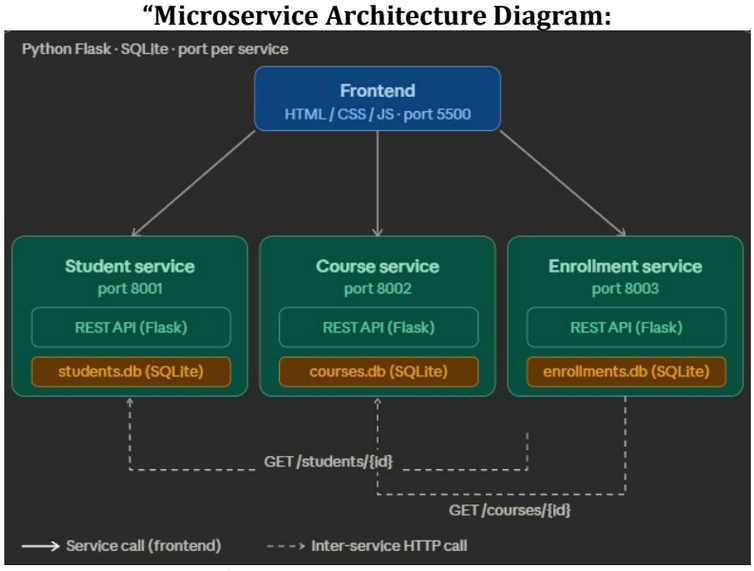
This exercise divides into two parts. Part A involves monolithic system using Node.js and Express with all three features; student, courses, and enrollments handled by a single application with in-memory storage. A single-page HTML/CSS/JavaScript frontend was served directly by the Express server.

"Monolithic Architecture Diagram"



Part B involved splitting the system into three independent microservices using Python Flask. Each service was assigned a dedicated port: Student Service on port 8001, Course Service on port 8002, and Enrollment Service on port 8003. Each service maintained its own SQLite database managed through Flask-SQLAlchemy, and the services communicated through HTTP request using the requests library.

"Microservice Architecture Diagram:



III. EXPERIMENTAL FINDINGS / OBSERVATIONS

Observations during tests on how each architecture show different findings. In microservices implementation intentional stoppage of Student Service caused the student-related sections of the interface to become unavailable the other services are still operating without interruptions. This confirmed the boundaries successfully contained the failure. In monolithic architecture, any disruption on any single component renders the system unusable, since all the features shares the same process and data layer.

Another difference on the two architecture is how removal behaves on the system. In monolithic architecture removing a student record automatically cascaded to its associated enrollment records within a single database transaction, this ensures the referential integrity was preserved. On the other hand, microservice architecture have no such mechanism, deletion of a student from the Student Service left orphaned enrollment records in the Enrollment Service's database. This is an inherent limitation of distributed architectures, where cross-service transactional guarantees require coordination to such domain events or compensating transactions.

In the observance of communication in between services also arises a difference. In monolithic architecture enrollment data synch instantly since all records were in the same data storage, while the microservices architecture shows a measurable delay that would compound as the number of enrollment rows increases. Lastly, in the setup effort and the deployment complexity between the two architecture shows up. In microservices each independent services requires their own entry point, database configuration, route definitions, environment file, and dependency list. With this all had to be started in correct order across separate terminals. In monolithic architecture consolidates all of the data and services into a single project. This difference in effort was the most immediate in between implementations during development.

IV. LAB DISCUSSIONS / COMPUTATIONS

Criteria	Monolithic	Microservices
Ease of Development	Single codebase, one DB	3 services, 3 DBs, HTTP wiring
Deployment	One command to start all	Multiple terminals, order matters
Scalability	Entire app scales as one	Each service scales independently
Failure Impact	One bug affects everything	Failures isolated per service
Performance	Zero HTTP overhead	Network latency on every API call

The table above summarizes the key tradeoffs identified across both implementations. The monolithic architecture demonstrates clear advantages in development speed, deployment simplicity, performance, and data consistency. All operations completed within a single process and database, eliminating network overhead and enabling transactional guarantees that the distributed version cannot provide without additional infrastructure. The microservice architecture gets ahead on the areas that matter most at scale: independent scalability per service, and the containment of failures to only the affected boundary. The absence of a cross-service cascade mechanism is a real tradeoff one that would need to be addressed through domain events in a production system.

V. CONCLUSIONS

The exercise demonstrated that the choice between monolithic and microservice architecture is fundamentally determined by the scale and growth expectations of the system. For a project of this scope, the monolithic approach is the more practical and efficient choice it is faster to develop, simpler to deploy, and delivers better runtime performance with no inter-service overhead. The same feature set built as three separate services required significantly more time and configuration to reach the same functional outcome.

The microservices architecture demonstrates its value in areas that only become relevant at production scale. The Enrollment Service's reliance on both the Student and Course Services is a realistic model of how dependent operations work in large systems. As user volume grows, the ability to scale the enrollment database independently without forcing the student and course databases to grow alongside it represents a meaningful operational and cost advantage. In cloud environments where infrastructure is billed by resource consumption, this kind of isolated scaling directly reduces operating costs.

Resilience is another area where the microservices model proves its worth at scale. A failure in any part of the monolithic application takes the entire system offline. In the microservices version, a downed service affects only its own boundary students and courses remain readable even when the enrollment service is unavailable. For any institution that depends on uninterrupted access to its registration system, this fault isolation significantly reduces the blast radius of incidents. The lack of a built-in cascade delete mechanism is a known limitation that would need to be solved through event-driven coordination, but it is a solvable problem rather than a fundamental barrier.

The tradeoffs in complexity and performance are real but manageable. The additional scaffolding required for microservices is a fixed cost paid once at setup, not a recurring burden. The inter-service latency observed during enrollment loading is addressable through bulk-fetch optimizations, which were applied during testing and reduced the overhead significantly. On balance, microservice architecture is the more defensible long-term investment for any system expected to grow in users, traffic, or team size — while the monolith remains the correct choice for systems where simplicity and time to delivery take priority.

References

Fowler, M., & Lewis, J. (2014). Microservices: A definition of this new architectural term. martinfowler.com. Retrieved from https://martinfowler.com/articles/microservices.html

Newman, S. (2021). Building microservices: Designing fine-grained systems (2nd ed.). O'Reilly Media.

Pallets Projects. (2024). Flask documentation (3.0.x). Retrieved from https://flask.palletsprojects.com/

Python Software Foundation. (2024). Python 3 documentation. Retrieved from https://docs.python.org/3/

Richardson, C. (2018). Microservices patterns: With examples in Java. Manning Publications.